Module 2: From One Agent to a System

We already know that an AI agent works toward a goal by interacting with external tools and systems to complete tasks.

But here's the interesting part: **the model doesn't directly interact with those tools.** Instead, it uses a mechanism called **tool calling** to request an action. The surrounding application executes the tool and sends the result back to the model.

So, the model decides what needs to be done, while the application handles the actual execution.

How Tool Calling Works

Before anything happens, the developer defines the tools available to the model.

Each tool definition contains three main components:

1. **Name:** What the tool is called.
2. **Description:** What the tool does and when it should be used.
3. **Input Schema:** What parameters the tool expects and their required format.

Important: The model sees the tool definitions, not the actual implementation behind them.

Step-by-Step Example: Refund Agent

Let's say a user tells the agent, *"My order is broken, and I want a refund."*

Here's what happens behind the scenes:

1. **User request:** The user asks for a refund because their order arrived damaged.
2. **Model receives context:** The model sees the user's request along with descriptions of the tools it can use.
3. **Tool selection:** Based on those descriptions, the model decides which tool is appropriate and generates a structured tool call containing the tool name and required parameters, such as `order_id`.
4. **System receives the request:** The application surrounding the model intercepts the tool call.
5. **Validation and safety checks:** Before executing anything, the system checks whether:
    - The request follows the expected schema.
    - The agent has permission to use the tool.

    - The provided values are valid and make sense.
    - Any required authorization or approval has been obtained.
6. **Tool execution:** Once the checks pass, the application executes the tool, which communicates with the order management system.
7. **Tool response:** The order system returns the result, such as the order details or refund status.
8. **Model receives the result:** The application adds the tool's response to the model's context.
9. **Final response:** The model uses the returned information to decide whether another action is needed or to generate a response explaining the outcome to the user.

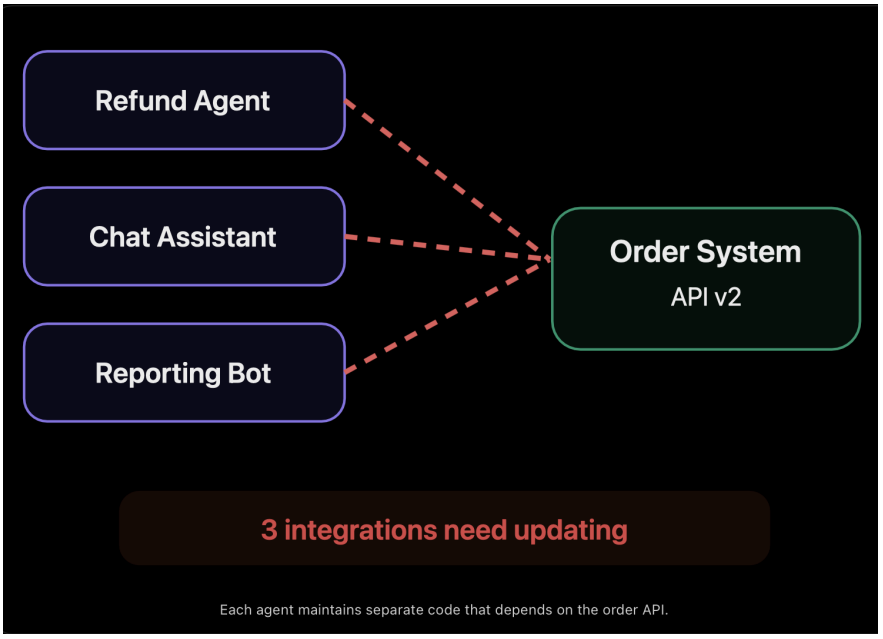
Here, the `order_system` tool needed by the Refund Agent was manually integrated by a developer.

But what happens when multiple AI agents need to use the same tool?

If every agent has its own separate connection to the Order System, we'd have to update each integration whenever the Order System changes.

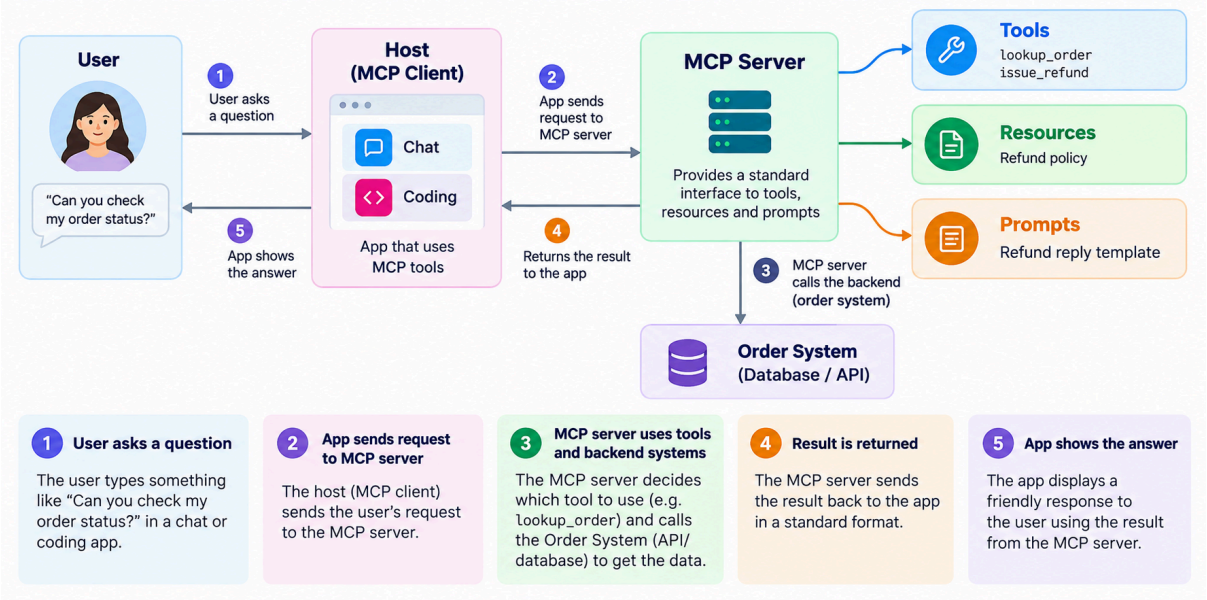
For example, if the API changes from `order_id` to `orderId`, every agent depending on the old format might break.

As the number of agents grows, maintaining all these individual integrations becomes a hassle. There has to be a better way to share tools across multiple agents without repeatedly updating the same integration.



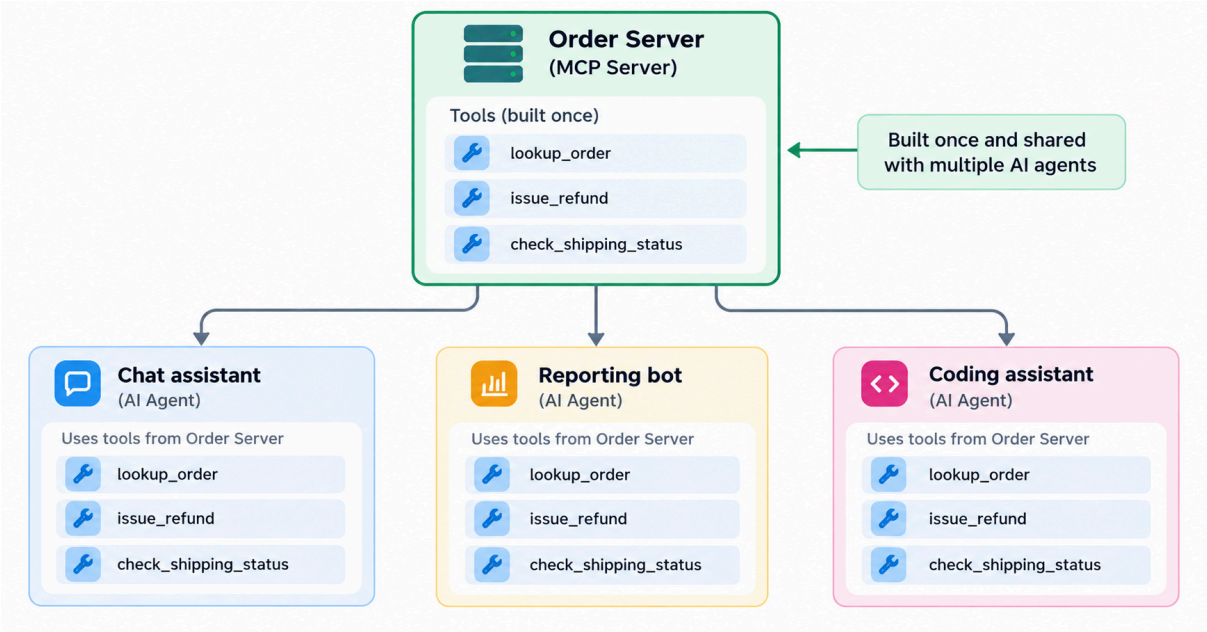
Solution: MCP

MCP is an interface where it standardizes the tool calling between client(AI agent) and tool(Order_System). The client doesn't have to worry about the implementation of tool.



1. Model: Receives the available tool descriptions and decides to call lookup_order for order 1042.
2. Host + MCP Client: Receives the model's tool request and forwards it to the MCP server.
3. MCP Server: Executes the lookup_order tool and communicates with the Order API.
4. API: Sends a request to the Order System to retrieve order 1042.
5. Order System: Fetches the order details from the database.
6. Response: The result (delivered, $49) travels back through the API, MCP server, and host to the model, which uses it to answer the user.

MCP doesn’t replace API’s, It just lets the Agents connect to the external API’s in one standard way.



Problems with MCP

1. Security Risks

What if an MCP server is malicious or compromised?

Since the server provides tool descriptions and responses, it could expose an agent to malicious instructions or unsafe tools. If the agent blindly trusts them, it might perform unintended actions or expose sensitive information.

2. Context Window Overhead

Every tool exposed to the model comes with a name, description, and input schema.

Imagine connecting an agent to multiple MCP servers, each providing dozens of tools. If all those tool definitions are loaded into the model's context, they consume a significant number of tokens.

This leaves less room for conversation history and other important information, potentially increasing costs and making tool selection harder.

3. Missing Business Rules and Context

Every team has its own business rules and workflows, but these aren't always captured in MCP tool descriptions.

Let's take our **Refund Agent** as an example.

Suppose the tool description simply says:

`issue_refund` — Issues a refund for a customer order.

But your company has additional rules:

- If the refund amount exceeds $100, the customer must upload a photo as proof.
- If the order was a gift, the refund process is different.
- Certain refunds require approval before processing.

An Agent Skill is a folder containing instructions, guidelines, and optional resources that teach an AI agent how to perform a specific task.

At the center of the folder is a file called `SKILL.md`, which contains the main instructions.

MCP — How does the agent access and interact with external tools?
Agent Skills — How should the agent approach and complete a task

Key takeaway: MCP standardizes tool integrations, but it doesn't automatically solve security risks or context window limitations. These still need to be handled by the application.